

Grounding the Tutor

How we are adding retrieval to the offline AI Tutor, what we chose for each layer, and — more usefully — the four things we believed at the start that measurement proved wrong.

Status revised after review · five blocking gaps closed

Scope English + Hindi + Marathi, mixed library Target CPU-only offline laptop

Read this if you are on the team: the stack table below, then §2 (what changed and why). Read this if you are implementing: all of it — §4 and §5 are written to be executed from.

Every number in this document was measured on the real stack — the models we actually have pulled, the SQLite build we actually ship, at the corpus size we actually expect. Nothing here is quoted from a benchmark blog.

WHAT CHANGED IN THIS REVISION

An architecture review found five defects that would have shipped: the tutor could no longer decline to answer, the lexical leg returned nothing for any question phrased as a question, no component decided what language the query was in, the context budget was counted in a unit that Devanagari does not respect, and the embedding model had been made permanent by accident. § 03 records all five and the decision taken on each; the stack table, the query diagram and the phases below already reflect them.

§ 01 The stack

EVERY LAYER, DECIDED

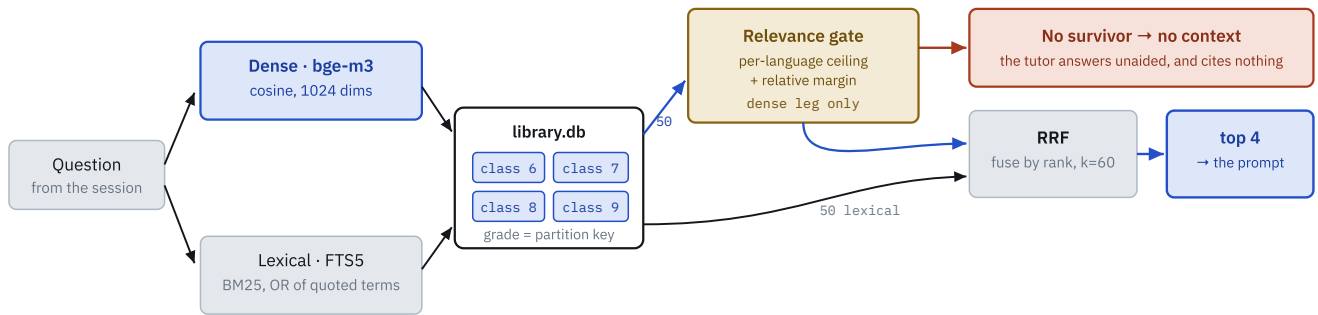
CHOSEN CONFIGURATION		ALL VERIFIED LOCALLY
LAYER	CHOICE	WHY THIS ONE
Vector store	sqlite-vec 0.1.9 · vec0	One <code>.db</code> file. Curriculum built centrally; students write their own books into the same file on-device.
Index	flat scan + grade partition	7.7 ms at curriculum size. ANN saves ~9 ms of a ~2,850 ms answer, and is worse for a store students mutate.
Embeddings	bge-m3 · 1024 dims · cosine	The only model tested that ranks correctly on Hindi→English <i>and</i> Marathi→English.
Lexical search	SQLite FTS5 · trigram	Same file, zero new dependencies. Trigram is the only tokenizer that survives Devanagari.
Fusion	Reciprocal Rank Fusion, k=60	Rank-based, so BM25 and cosine never need a common scale.
Chunking	structural + breadcrumb	target 1200 / max 2000 / min 40 chars. Headings are hard cuts.

LAYER	CHOICE	WHY THIS ONE
Overlap	180 chars (15%), 0 at headings	Sentence-aligned. A heading is a topic boundary, so nothing carries across it.
Partition key	grade	Always known, never crossed. Measured 5.1× speed-up.
Filters	language soft · no subject filter	Subject is already inside every vector via the breadcrumb.
Query construction	built, never passed through	FTS5 puts an implicit AND between bare terms, so a whole question matches nothing . Terms are quoted and OR-joined.
Abstention	dense leg decides	Nothing survives the gate → no context at all. BM25 alone can never put a citation in front of a student.
Query language	from the session, not the text	The ASR and the UI toggle already know it. Sniffing is a fallback, and an unsure fallback picks the <i>loosest</i> ceiling.
Context budget	tokens, not characters	Devanagari costs 2–4× the tokens per character. Passages are added whole; k falls before a passage is cut.
Model migration	model-scoped vec0 table	Re-embed in the background from stored text. The embedding choice is reversible, not a one-way door.
Compression	none	63.6 MB of vectors. The threshold for int8 is around 1 GB.
Reranker	none initially	A cross-encoder on this CPU costs more than the prefill it would save.
Generation	gemma2:2b (gemma3:4b configurable)	Both ground correctly. Gemma 3 writes better Devanagari at 1.35× decode and 2× RAM.

THE SHAPE OF THE WHOLE THING

A single SQLite file per device, containing the textbook text, its metadata, a dense vector index and a full-text index. No server, no network, no separate index artefact. That constraint — *it has to be one file you can copy onto a laptop* — drove more of the decisions below than any retrieval-quality argument did.

ONE QUESTION, TWO RETRIEVERS, ONE FILE · THE GATE DECIDES WHETHER THERE IS AN ANSWER



10 ms

both legs read the same file; only the dense leg is partitioned

relevance is decided before ranks replace distances

The query path. Both retrievers read the same file, so hybrid search adds no deployment complexity. The gate sits on the dense leg **before** fusion, because RRF replaces distances with ranks — and because a lexical match is evidence of shared spelling, not of relevance. When nothing clears the gate the model is asked without context rather than handed the four least-bad passages.

§ 02 Four things we had wrong

THE DECISIONS THAT CHANGED WHEN WE MEASURED

The plan we started from was reasonable and mostly derived from general RAG advice. Four parts of it did not survive contact with this specific stack. These are the ones worth your attention, because each was a decision we would otherwise have shipped.

OVERTURNED · 01

“Index: HNSW, tune M and ef_construction”

sqlite-vec has no HNSW. Passing any ANN table option is rejected outright (Unknown table option: index), and there is no approximate-search code anywhere in the package. Choosing HNSW would mean replacing the vector store — and with it the single-file property the whole offline distribution depends on.

It also turned out to be unnecessary. Because `grade` is a partition key, sqlite-vec physically shards the table and a query only scans the student's own grade:

NO PARTITION

51.2 ms

scans all 15,000

GRADE PARTITION

10.1 ms

scans 3,000 · 5.1× faster

FOR CONTEXT

2,800 ms

prefill of 4 passages

HNSW would be optimising a 10 ms step sitting in front of a 2,800 ms one. **Decision: flat scan, no ANN, and the M / ef_construction tuning task is deleted.** Revisit past roughly 100k chunks.

OVERTURNED · 02

“Keep nomic-embed-text”

This is the serious one. On the Hindi question “ऊतक क्या है?”, nomic-embed-text scored an **off-topic** passage about the capital of India *higher* than the passage that actually defines a tissue. A negative margin means Hindi retrieval is not merely weak — it is worse than random, and it fails silently.

RELEVANCE MARGIN — RELEVANT MINUS OFF-TOPIC, HIGHER IS BETTER						
MODEL	DIMS	HINDI→HINDI	HINDI→ENGLISH	MARATHI→ENGLISH	ENGLISH	INGEST / BOOK
nomic-embed-text	768	-0.021	—	—	good	1.8 min
bge-m3	1024	+0.363	+0.189	+0.201	+0.551	6.1 min
granite-embedding:278m	768	+0.266	+0.081	+0.080	+0.441	2.0 min
embeddinggemma	768	+0.204	ranks wrong	+0.133	+0.495	2.5 min

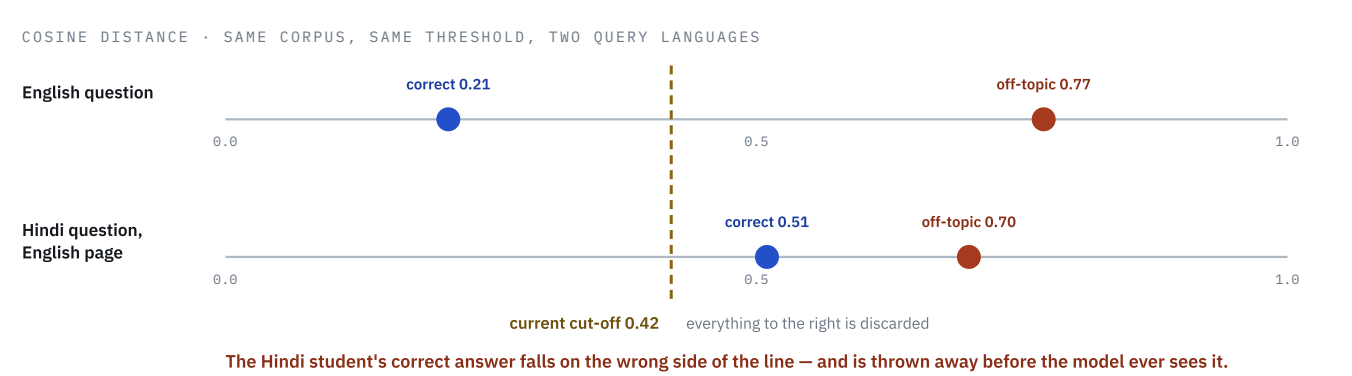
Decision: bge-m3. The trade-off is real — 1024 dims instead of 768, and 3.4× the ingestion time. We accept it because *ingestion happens once, centrally*, on a fast machine, and the result is shipped as a file. The cost lands where we can afford it.

Granite was the tempting middle option at 768 dims and 2 minutes per book, but its cross-lingual margin of +0.08 is too narrow to set a reliable threshold against — relevant and off-topic passages sit almost on top of each other.

OVERTURNED · 03

“One rag_max_distance = 0.42”

A single global relevance threshold cannot work across languages, because the distance scale itself shifts with the query language. With bge-m3, a *correct* hit sits at distance 0.21 for an English question but 0.51 for a Hindi question against the same English page.



Why one number cannot serve three languages. The English scale is wide and easy to split. The cross-lingual scale is compressed and shifted right, so the threshold tuned on English silently discards every correct Hindi result.

Decision: replace the single constant with two mechanisms — a **relative gate** (keep hits within +0.12 of the best hit, which normalises away the language offset because every candidate for one query shares its query language) and a **per-query-language**

ceiling that rejects everything when even the best hit is too far. Starting values `en 0.45 · hi 0.62 · mr 0.62`, to be re-measured against the golden set, not treated as constants.

CONFIRMED · 04

“Is gemma2:2b still the right generator?”

Tested on a full four-passage RAG prompt with a Hindi question against English passages — the mixed-library case. Both viable models stayed grounded, including on a probe that required recovering a specific term from the third passage.

MODEL	HINDI ANSWER FROM ENGLISH CONTEXT	GROUNDED	DECODE	RAM
gemma2:2b	...sclerenchyma ऊतक से बना होता है। <code>term left in Latin</code>	yes	14.1 tok/s	1.6 GB
gemma3:4b	...स्कलेरेन्काइमा ऊतक (Sclerenchyma tissue) से... <code>transliterates + glosses</code>	yes	10.0 tok/s	3.3 GB
qwen2.5:1.5b	fluent-looking Devanagari, semantically meaningless	no	23.2 tok/s	1.0 GB

Gemma 3 handles technical vocabulary the way a Hindi-medium textbook does — transliterate, then gloss the English in brackets. That is genuinely better for the student. But we are now also holding bge-m3 resident, so the pair costs **4.5 GB against 2.8 GB**.

Decision: keep gemma2:2b as the default, make the choice configurable, and settle it on the real hardware. If the fleet has 16 GB, Gemma 3 is the better student experience.

Worth noting the general lesson: *good retrieval lowers the bar for the generator*. With the right passage in context the model is extracting and translating rather than recalling, which is exactly what keeps a 2B model adequate here.

§ 03 Five gaps the review closed

DEFECTS IN THIS DESIGN, NOT IN THE PLAN IT REPLACED

§ 02 records four beliefs that measurement overturned. These five are different in kind: they are faults in the design on this page, found by reviewing it against the branch it will be implemented on. Each one fails *silently* — dense retrieval keeps working, answers keep looking plausible, and every check in the original verification list still passes. That is what makes them worth a section rather than a footnote.

CLOSED · 01

The tutor had lost its ability to say nothing

The gate was to be applied to the dense leg before fusion — correct, but it left the BM25 leg with no relevance test anywhere in the pipeline. Today `rag_max_distance` is the only thing standing between an uncovered question and four confidently cited passages. Once hybrid search lands, a question the library does not cover correctly empties the dense leg, and RRF still has fifty lexical candidates to rank. The top four reach the prompt with chapter and page attached.

An answer that is wrong is a bad answer. An answer that is wrong *and carries a citation to the student's own textbook* is a different category of failure, and it is the one this design would have introduced.

DECISION

The dense leg decides *whether* there is an answer; the lexical leg only decides the order. If nothing survives the gate, `retrieve()` returns `[]` and BM25 is never consulted. If something does survive, every passage in the final four must either clear the gate itself or be an immediate neighbour, in the same document, of one that did — which keeps a split definition retrievable without letting an unrelated lexical hit ride in on a good one.

The verification list gains the test that would have caught this: an out-of-syllabus slice, in every language, whose **expected result is zero chunks**.

CLOSED · 02

A question, handed to FTS5, matched nothing at all

The plan said to run BM25 with fifty candidates and never said how the `MATCH` string is built. FTS5 puts an implicit **AND** between bare terms, so every word of the question has to appear in the passage. Under `trigram` it is worse: a term shorter than three characters produces no tokens at all and silently zeroes the whole conjunction. Measured against the real tokenizer:

```
MATCH 'What is a tissue'      → []
MATCH 'Explain tissue'       → []
MATCH 'define tissue please' → []
MATCH 'tissue'                → 3 hits  the only shape that works
```

So the lexical leg would have contributed nothing to any question phrased as a question. Dense retrieval carries the system, the answers still look right, and the original check passed — because it tested a bare term.

DECISION

The query is built, never passed through. Strip punctuation, drop stopwords and any token under three characters, double embedded quotes, wrap each surviving term in quotes and join with `OR`. Append the whole question as one quoted phrase — under trigram that is a substring search, and it is exactly the exact-match behaviour hybrid was added to buy. A unit test asserts that a natural-language question returns more than zero rows.

CLOSED · 03

Nothing decided what language the question was in

The per-language ceiling is the mechanism that replaces the single threshold, and the design never said where the language comes from. Script detection is the obvious implementation, and it fails on the most common typed input in this market. A student typing “*utak kya hai*” is read as English, takes the strictest ceiling of 0.45, and has a correct hit at 0.51 discarded — which is the failure in **Overtuned · 03**, returning through the one path nobody measured.

Two smaller holes sit beside it. Hindi and Marathi share Devanagari, so script detection can never separate them; they happen to share a ceiling today, which hides it. And this work lands on `aibharat-stt`: for a spoken question the language is already settled by the ASR, and nothing connected the two.

DECISION

The language comes from the session — the ASR selection or the UI toggle, both of which already know it. Text sniffing is a fallback, and when the fallback is not confident it applies the **most permissive** ceiling rather than the strictest: a false accept costs one noisy passage, a false reject costs the entire answer and says nothing. Latin-script Indic is its own bucket with its own measured ceiling, rather than being swept into `en`.

The golden set gains romanized Hindi and Marathi questions. As written it tested Devanagari input only, which is the half of the problem that was already working.

CLOSED · 04

The context budget was counted in a unit Devanagari does not respect

Chunk bounds are in characters and phase 03 raises the maximum to 2,000 of them. `num_ctx` is 4096 and every request sends it. Indic scripts cost roughly two to four times more tokens per character than English, so four 2,000-character Devanagari passages, plus breadcrumbs, system prompt and replayed history, can exceed the window. Ollama then truncates from the front — discarding the system prompt and the first passage without raising anything.

The same blindness runs through the latency figures. The **~2,800 ms prefill** quoted in § 02 was measured on English passages; the same four passages drawn from a Hindi textbook are two to four times the tokens. The argument against HNSW survives — the ratio only widens — but the absolute number does not.

DECISION

The prompt is assembled against a **token** budget, and `k` falls before a passage is cut: passages are added whole until the budget is reached, because three whole passages beat four half ones. Counting uses the model's own tokenizer where it is reachable and a per-script characters-per-token constant that errs low where it is not. `num_ctx` rises to fit the worst case, and resident memory is re-measured with it — the KV cache scales with the window, so the 1.6 GB and 3.3 GB figures in § 02 do not hold at a larger one.

Prefill is re-measured on Devanagari source passages before it is quoted again.

CLOSED · 05

The embedding model was never actually a one-way door

The section below stated that the model cannot be revisited once the curriculum index ships, and treated that as a property of the store. The premise is right — vectors from two models are not comparable, and mixing them fails silently. The conclusion is not, because `chunks.text` is already on every device. Re-embedding reads data that is present locally; it is a background job, not a redistribution.

The cost of the mistake was not technical. It made the highest-stakes decision in the project urgent, and forced it to be settled against the thinnest evidence in the document — a twelve-passage corpus — under a deadline that did not need to exist.

DECISION

The vec0 table is named for its model — `chunk_vectors_bge_m3` — and `meta` keeps its stamp, which is what it is good at. A re-embed job reads `chunks.text`, fills a new table in the background, and cuts over on completion while the old table goes on serving queries. Cost is one overnight run per device. That is the price of being able to change your mind, and it is far below the price of not being able to.

Everything downstream relaxes: `embeddinggemma` stops having an expiry date, and the on-device ingestion timing in § 04 becomes something to measure rather than something to race.

§ 04 Students add their own books

A CONSTRAINT ON THE STORE, NOT A FEATURE ON TOP OF IT

Because a student can add a textbook themselves, the store has to accept writes on a 15 watt laptop with no rebuild step. That removes the “build centrally, ship a file” assumption for student content — and it is what finally settles the index question.

FLAT SCAN AS THE LIBRARY GROWS · SQLITE-VEC, 1024 DIMS, PER GRADE PARTITION			
BOOKS IN THE GRADE	CHUNKS IN PARTITION	K=50 QUERY	VERDICT
6 — shipped curriculum	4,400	7.7 ms	invisible
12	6,600	10.5 ms	invisible
25	11,000	19.5 ms	invisible
50	22,000	50.5 ms	fine
100	44,000	99.7 ms	noticeable

Adding a 440-chunk book costs **118 ms** of index write. Deleting one costs **236 ms**. No rebuild, no tombstones, no retraining.

WHY THIS RULES OUT HNSW RATHER THAN ARGUING FOR IT

HNSW would take 7.7 ms down to roughly 1 ms — **9 ms saved against a ~2,850 ms answer, or 0.3%**. Even at an implausible 100 books per grade it is 3.4%. The latency lives in prefill, not search.

More importantly it is *operationally worse* for a store the student mutates: incremental inserts degrade graph quality over time, and deletes are soft, needing periodic compaction. LanceDB's IVF is worse again — its centroids need retraining as student additions shift the data distribution. Flat scan has none of these failure modes. **The distribution model stands.**

The cost this does create: on-device embedding

bge-m3 runs at roughly **850 ms per chunk** on CPU, and batching does not help — Ollama processes sequentially (batch 1: 913 ms/chunk; batch 64: 899 ms/chunk). A 440-chunk book is **~6.5 minutes on a fast Mac, projecting to 20–27 minutes on the 15 watt target**.

So there are two ingestion paths sharing one model:

- **Curriculum** — pre-built centrally, shipped as the `.db` file. Unchanged.

- **Student additions** — embedded on-device into the same file, using the background job with progress polling that `ingest.py` already implements. The student keeps using the tutor while it runs.

SAME MODEL PER INDEX – BUT THE INDEX CAN BE REBUILT

A student-added book **must** use the same embedding model as the vectors it is joining: vectors from two models are not comparable, and mixing them fails silently rather than loudly. The `meta` stamp enforces that, and the model-scoped table name (**Closed · 05**) makes it structural rather than a convention.

What this does *not* mean is that the choice is permanent. Because `chunks.text` lives on the device, a different model is a background re-embed into a second `vec0` table with a cutover at the end — roughly one overnight run per device, with the tutor answering throughout. So the 20-minutes-per-book question below is one to measure carefully, not one to race the curriculum ship date.

A broader evaluation justified paying the cost — ten Hindi and Marathi questions against a twelve-passage English corpus:

CROSS-LINGUAL RETRIEVAL ACCURACY VS INGESTION COST				
MODEL	RECALL@1	RECALL@3	MS / CHUNK	440-CHUNK BOOK
bge-m3	10/10	10/10	~850	6.5 min
granite-embedding:278m	8/10	9/10 1 unanswerable	~275	2.0 min
embeddinggemma	8/10	10/10	~336	2.5 min

Granite is three times faster but misses one question *entirely* — unanswerable, not merely mis-ranked. If the on-device wait proves intolerable on real hardware, `embeddinggemma` is the documented fallback, at a measured cost of two questions in ten dropping out of first place.

§ 05 The decisions we kept, and what they cost

NOTHING HERE IS FREE

DECISION REGISTER		
DECISION	REJECTED ALTERNATIVE	TRADE-OFF WE ACCEPTED
Hybrid search + RRF	Dense only	A second index to keep in sync inside the same file. Buys exact-term matching for formulas, proper nouns and chapter references that embeddings blur. Note it contributes little cross-lingually — a Hindi query shares no tokens with an English page, so RRF degrades gracefully to dense-only there.
FTS5 <code>trigram</code> tokenizer	Default <code>unicode61</code>	A larger text index and no word semantics. Non-negotiable though: <code>unicode61</code> shatters Devanagari conjuncts — कार्य becomes the single letters क, र, य, which are useless for BM25. Trigram indexes every script correctly.
Structural chunking	Recursive splitter	Effectively none — our splitter <i>is</i> a recursive splitter (paragraph → sentence) with a heading level on top. When no heading is detected it degrades to exactly the recursive behaviour, so this is a strict superset.
Breadcrumb enrichment	Plain chunk text	~30 characters per chunk. Puts the chapter's vocabulary into every vector, which is what makes paragraphs full of “they” and “these cells” retrievable at all.
Overlap 15%, zero at headings	Uniform overlap	18% more chunks to store and embed. Carrying text across a heading would mix two topics in one vector <i>and</i> file the chunk under the wrong chapter in the citation shown to the student.

DECISION	REJECTED ALTERNATIVE	TRADE-OFF WE ACCEPTED
Grade as partition key	Grade as a filter	Cross-grade search would need a fan-out, which we never want. In exchange the search space shrinks 5×, and a post-filter can never quietly return fewer than <code>k</code> results.
No subject filter	Filter on subject	Slightly noisier candidates. But subject filtering measured only 1.9 ms of saving, and a mismatch between the UI's subject label and the uploader's free-typed one can zero out recall entirely. The breadcrumb already encodes subject in every vector — softly, and it can never return nothing.
No reranker yet	Cross-encoder	Lower precision at rank 1–4 than we could have. Revisit only once recall@50 is measured; on this CPU a reranker must save more prefill than it costs, which is a genuinely close call.
No compression	int8 / PQ	None — 63.6 MB of vectors is nowhere near the ~1 GB where quantization starts paying. Optimising this would be pure ceremony.

THE ONE THAT WILL SURPRISE PEOPLE

Dropping the subject filter feels wrong — it looks like giving up precision for nothing. The reason it is safe is that `Chunk.embedding_text()` already prepends `Class 9 > Science > Tissues` before embedding. Subject is therefore *inside the vector*, ranking Science chunks higher for Science questions without ever being able to return an empty result set. We built a soft filter and then almost added a hard one on top of it.

§ 06 Implementation

SEVEN PHASES · WRITTEN TO BE EXECUTED FROM

00 Cut a new feature branch

`feature/latency-optimization-rag` stays **untouched** as the English-only reference. Cut a new branch from `aibharat-stt` and merge the RAG work into it — a merge only writes to the target, so the source branch is unaffected.

```
git checkout aibharat-stt && git checkout -b feature/multilang-rag
git merge feature/latency-optimization-rag
```

Both share merge-base `650ee47` — 3 commits ahead and 2 ahead respectively. Conflicts should be additive only.

`config.py` · `requirements.txt` · `app/main.py`

01

Store: language column + FTS5

Bump `SCHEMA_VERSION` to 3. Add `language` to `documents` and as a `vec0` metadata column — *not* a second partition key, since `grade × language × subject` would over-partition into fragments too small to help.

```
create virtual table chunks_fts using fts5(
  text, content='chunks', content_rowid='id', tokenize='trigram');
```

Write FTS rows inside the same transaction as `add_chunks()`, and extend `delete_document()` to remove them — it already deletes vectors by hand because `vec0` ignores foreign keys, and external-content FTS5 tables need the same care.

Create the vector table under a name that carries its model — `chunk_vectors_bge_m3` — and record the active table in `meta`. It costs one line now and is what makes the embedding model reversible later (**Closed · 05**); retrofitting it after an index ships is a schema migration on every device.

`services/rag/store.py`

02

Swap the embedding model

`rag_embedding_model: "bge-m3"`, `rag_embedding_dims: 1024`. `bge-m3` needs no task prefix — leave `_needs_prefix()` in place so the nomic path stays correct if anyone switches back. The existing `_check_or_stamp_meta()` guard will correctly refuse to open a v2 store, which is exactly what we want.

Then write the re-embed job while the schema is still moving: read `chunks.text`, fill a second model-scoped table in the background using the progress-polling machinery `ingest.py` already has, and report `meta` when it completes. It is the same job the student-upload path runs, pointed at content already in the file.

`services/rag/embeddings.py · config.py · services/rag/ingest.py`

03

Chunk bounds and heading detection

Split the single `chunk_chars` into target 1200 / max 2000 / min 40. Paragraph packing uses *target*; the long-paragraph sentence split uses *max*; the existing `>= 40` filter becomes *min*. The gap lets a section run slightly long to stay whole rather than being cut at character 1,201.

Then fix two defects in `looks_like_heading()`: the title-case branch currently makes *any* two capitalised words a heading, so figure labels like “Xylem Vessels” become spurious hard cuts; and Devanagari has no case, so only the numbered-heading regex can ever fire for Hindi and Marathi books.

These bounds govern *splitting* only. They are characters because splitting is a text operation, and they must not be mistaken for a context budget: 2,000 characters of English is about 500 tokens and 2,000 characters of Hindi can be three times that, which is why the prompt is assembled against tokens in phase 04 (**Closed · 04**).

`services/rag/chunking.py · services/rag/pdf_text.py`

04 Hybrid retrieval + RRF

Run dense and BM25 over the same grade partition, 50 candidates each, fuse with RRF at $k=60$. **Delete the subject-filter retry** — subject leaves the query entirely. `k_context` stays 4. Keep `retrieve()` non-raising: retrieval is additive, and a missing library must degrade rather than fail.

Build the MATCH string; never pass the question through (Closed · 02). Strip punctuation, drop stopwords and tokens under three characters, double any embedded `"`, quote each surviving term and join with `OR`, then append the full question as one quoted phrase for exact matches. Test that a natural-language question returns more than zero rows — without that test this regresses the first time someone simplifies the builder.

Then assemble the context block against a **token** budget rather than `k_context` alone (**Closed · 04**): add passages whole until the budget is reached, and drop from four toward two rather than truncating a passage. Raise `num_ctx` to fit the worst Devanagari case and re-measure RSS with it.

`services/rag/retrieval.py · services/ollama_client.py · config.py`

05 Language-aware relevance gate

Replace `rag_max_distance` with a relative gate (+0.12 from the best hit) plus a per-query-language ceiling. Apply it to the dense leg *before* fusion, since after RRF you are ranking by fused position and the distances are gone.

Take the query language from the session — the ASR selection or the UI toggle — and sniff the text only as a fallback; when the fallback is unsure, apply the loosest ceiling, not the strictest (**Closed · 03**). Give Latin-script Indic its own bucket.

Finally, wire the abstention rule (**Closed · 01**): an empty dense leg after gating returns `[]` without consulting BM25, and each of the final four must clear the gate itself or neighbour a passage that did.

`services/rag/retrieval.py · config.py`

06 Evaluation harness

`POST /api/library/search` already runs retrieval without the model — build on it. A golden set of 30–50 real questions *per language*, each with its expected chunk; a report of `recall@50`, `precision@4`, `MRR` and `nDCG@4` **split by query language**; and a calibration mode printing on-topic against off-topic distance distributions so the phase 05 ceilings are measured rather than guessed.

Two slices matter as much as the ranked ones: **romanized** Hindi and Marathi questions, which are the input the ceilings are most likely to mishandle, and an **out-of-syllabus** set whose expected result is zero chunks — the only test that can tell a working gate from one that has been tuned into permissiveness.

`routers/library.py · scripts/`

§ 07 How we will know it worked

VERIFICATION

1. Ingest one English and one Devanagari textbook. Confirm `/health` reports the library available and `meta` stamps `bge-m3 / 1024` against the model-scoped table.

- 2. Search an English question against the English book — the control.
- 3. **Search a Hindi question against the English book.** This is the case that fails today and the single most important check in this list.
- 4. **Ask the same question in romanized Hindi** and confirm it retrieves the same passages. This is the check that would have caught **Closed · 03**.
- 5. Confirm a *natural-language* question — not a bare keyword — returns more than zero BM25 rows, and that BM25 alone still retrieves an exact Devanagari term (कार्य).
- 6. **Ask something the library does not cover, in each language.** The tutor must answer unaided and cite nothing. If a citation appears, the gate is not doing its job.
- 7. Count the prompt's tokens at `k=4` against a Devanagari book and confirm it fits inside `num_ctx` with the system prompt intact.
- 8. Run the harness and record recall@50 per language, romanized included. This is the baseline every future change gets measured against — without it, later tuning is guesswork.
- 9. **The student-add path, on the target laptop.** Upload a book, confirm ingestion runs in the background with progress while the tutor stays answerable, and time it end-to-end.
- 0. Run the re-embed job on a populated library and confirm the cutover happens with the tutor answering throughout. This is what makes the embedding choice reversible; untested, it is not.
- 1. Delete a student-added book. Confirm its chunks, vectors *and* FTS rows all disappear, so no stale citation can survive.
- 2. On the *target laptop*: TTFT at k=4 vs k=2 *on Devanagari passages*, and total RSS with bge-m3 and the generator both resident at the raised `num_ctx`. Also worth pulling `gemma3:1b` — Gemma 3's multilingual training at a smaller size may beat gemma2:2b on Devanagari for less RAM.

DEFINITION OF DONE

A Hindi-speaking Class 9 student asks a question in Hindi, the tutor answers from the English textbook that the school actually owns, and cites the chapter and page. If that works offline on the target laptop, this is done. If it does not, nothing else on this page matters.

§ 08 Deliberately deferred

AND WHAT WOULD BRING EACH BACK

DEFERRED	REVISIT WHEN
HNSW / approximate search	Past ~100k chunks <i>per grade partition</i> — roughly 100 student-added books — where flat crosses 100 ms. Brings tombstone compaction with it.
embeddinggemma instead of bge-m3	Whenever on-device ingestion proves intolerable. No longer gated on the curriculum ship date — the re-embed job makes it a background migration. Costs 2 questions in 10 at rank 1.
Reranking	Once recall@50 is measured and good, but precision at rank 1–4 is not. Must save more CPU prefill than it costs.
Quantization	Around 1 GB of vectors. We are at 63.6 MB.
Subject as a hard filter	Only if a measured precision problem appears, <i>and</i> upload and session subjects are drawn from one controlled vocabulary.
gemma3:4b as default	Once RAM headroom on the real device is known. On a 16 GB fleet its Devanagari output is the better experience.
Indic-specialised generators	If both Gemma 3 1B and 4B disappoint. Cost is a model outside the Ollama mainline in every device image.

Grounding the Tutor · engineering decision record for the AI Tutor retrieval layer. All measurements taken against the local stack: sqlite-vec 0.1.9, SQLite 3.53.1, Ollama with bge-m3, gemma2:2b, gemma3:4b and qwen2.5:1.5b. Corpus figures assume ~15,000 chunks across five grades — five to six textbooks per grade at roughly 440 chunks each.

Every threshold in this document is a starting point to be measured against the golden set, not a constant to be trusted.

Revised after architecture review. § 03 records the five blocking defects found in the original and the decision taken on each; the stack table, the query diagram, the phases and the verification list were updated with them. The review also raised seven non-blocking findings — RRF weighting, the lexical leg's grade filter, phase ordering, corpus size behind the model choice, FTS5 delete ordering, the 40-character chunk floor and two disagreeing latency tables — which are not yet reflected here.